A Proof of Colton's Conjecture on Refactorable Numbers

Smooth–rough τ-decomposition, certified checkpoint chains, and a template tail

Keio

24 October 2026

Abstract

A positive integer k is *refactorable* (a *tau number*) if the number of its divisors $\tau(k)$ divides k. Let $T(n)$ be the number of refactorable integers in $[1, n]$. In 1999 Colton conjectured that $T(n) \geq \pi(n)/2$ for every n. In 2002 Zelinsky proved the inequality for all sufficiently large n, with the explicit bound $7.42 \cdot 10^{13}$ for possible counterexamples. The full statement has remained open and is recorded as open in the Formal Conjectures repository.

We prove the conjecture: $\pi(n) \leq 2T(n)$ for every $n \geq 1$, with equality only at $n = 7$. The proof does not use Zelinsky's bound. It rests on one observation: for $k < 2^{52}$, every prime factor of $\tau(k)$ is at most 47. Hence refactorability is decided by the 47-smooth part of k together with the exponent pattern of its 47-rough part. This *smooth–rough τ-decomposition* gives three things:

(i) an exact formula expressing $T(x)$ for $x < 2^{52}$ through values of the prime-counting function, which can be evaluated in sublinear time;

(ii) a certified lower bound $L(x) \leq T(x)$ for all x;

(iii) finite families of *templates*, integers whose products with large primes are always refactorable.

Combined with the Rosser–Schoenfeld bounds, (iii) settles every $x \geq 10^{14}$ through one explicit finite inequality. A monotone checkpoint principle turns finitely many values of L and π into proofs on whole intervals; 27 certified links close the range $[10^{11}, 10^{14}]$. The range $n \leq 10^{11}$ is covered by an exhaustive sieve, and independently by a 58-link chain starting at 10^6.

As a by-product we obtain three new terms of OEIS A111291:

$$T(10^{12}) = 26\,568\,322\,474, \quad T(10^{13}) = 251\,342\,379\,436, \quad T(10^{14}) = 2\,388\,671\,712\,980.$$

Each was confirmed by several different decompositions. All programs are listed in full in the appendix.

Contents

1 Introduction **2**

1.1 Refactorable numbers and Colton's conjecture 2

1.2 Main result 3

1.3 The method 3

1.4 New numerical data 4

1.5 Computer assistance and verifiability 4

2 Notation and classical inputs **5**

3 The smooth–rough τ-decomposition **5**

4 Monotone checkpoints and certified chains **7**

5 The tail $x \geq 10^{14}$ **8**

6 Proof of Theorem 1.2 **9**

7 Computations **10**
7.1 Exhaustive verification up to 10^{11} . . . 10
7.2 Evaluation of L and of the chains . . . 10
7.3 Exact values of T . . . 10
7.4 Cross-checks . . . 11
7.5 Timings . . . 11

8 Discussion **12**

A The certified chains **13**

B Source code **14**
B.1 Exhaustive parallel verification (`psieve.c`) . . . 14
B.2 Sequential verification with equality count (`sieve.c`) . . . 16
B.3 Linear-sieve cross-check of T and π (`check2.c`) . . . 16
B.4 Brute-force evaluation of L (`bruteL.c`) . . . 17
B.5 Lower bound L and certified chains (`checkpoints.cpp`) . . . 17
B.6 Exact formula for $T(x)$ with smoothness bound B (`exactT_B.cpp`) . . . 19
B.7 Template tail certificate (`tail.cpp`) . . . 21
B.8 Independent tail certificate (`tail_check.py`) . . . 23

1 Introduction

1.1 Refactorable numbers and Colton's conjecture

Let $\tau(k)$ denote the number of positive divisors of k. Kennedy and Cooper [5] called k a *tau number* if $\tau(k) \mid k$, and showed that these integers have natural density zero. Colton [1] rediscovered them with his automated theory-formation program HR and called them *refactorable*. They form OEIS sequence A033950 [6]:

$$1,\ 2,\ 8,\ 9,\ 12,\ 18,\ 24,\ 36,\ 40,\ 56,\ 60,\ 72,\ 80,\ 84,\ 88,\ 96,\ \ldots$$

Throughout we write

$$T(x) = \#\{1 \leq k \leq x :\ \tau(k) \mid k\}, \qquad \pi(x) = \#\{p \leq x :\ p \text{ prime}\} \qquad (x \geq 0).$$

Spiro [8] determined the order of magnitude of T: by his asymptotic formula, as recorded in [3],

$$T(x) = \frac{x}{\sqrt{\log x}}\,(\log\log x)^{-1+o(1)}.$$

In particular $T(x)/\pi(x) \to \infty$. The convergence is very slow, however: $2T(x)/\pi(x)$ is 1.095 at $x = 10^3$ and only 1.491 at $x = 10^{14}$ (Table 2).

Colton [1] conjectured that refactorable numbers are never less than half as frequent as the primes. We use the pointwise formulation stated by Zelinsky [10] and recorded in [3].

Conjecture 1.1 (Colton)**.** *For every positive integer n, $T(n) \geq \frac{1}{2}\pi(n)$; equivalently, $\pi(n) \leq 2T(n)$.*

Zelinsky [10] proved that for every fixed $c > 0$ one has $T(n) > c\,\pi(n)$ for all sufficiently large n. He also gave the explicit bound $7.42 \cdot 10^{13}$, beyond which Conjecture 1.1 holds. The finite range below this bound was left open. The Formal Conjectures repository [3] (snapshot of 23 October 2026) states the conjecture in Lean as

```
theorem colton_conjecture : ∀ n : ℕ, Nat.primeCounting n ≤ 2 *
                                countRefactorableNat n
```

It is tagged `research open`, with the remark that Zelinsky proved it beyond $7.42 \cdot 10^{13}$ "and the remaining range is open". There `countRefactorableNat n` counts the $k \in [1, n]$ with $\tau(k) \mid k$, so it is our $T(n)$.

A weaker asymptotic formulation, $T(x) \geq x/(2\log x)$ for all sufficiently large x, is recorded in [3] as solved, as a consequence of [10, 8].

Verifying the remaining range by brute force means computing $\tau(k)$ for every $k < 7.42 \cdot 10^{13}$. Our parallel sieve needs about 40 minutes on four cores just to reach 10^{11}, so this is a substantial computation (Section 7.5).

1.2 Main result

Theorem 1.2. *For every integer $n \geq 1$,*

$$\pi(n) \;\leq\; 2\,T(n),$$

with equality if and only if $n = 7$. In particular Conjecture 1.1 is true. The statement `colton_conjecture` of [3] also holds; for $n = 0$ both sides vanish.

Corollary 1.3 (Explicit asymptotic form)**.** *$T(x) > x/(2\log x)$ for every real $x \geq 2$, and this inequality fails for every $x \in (1, 2)$.*

Proof. For $x \geq 17$, Theorem 1.2 and Lemma 2.1(a) give

$$T(x) = T(\lfloor x \rfloor) \geq \tfrac{1}{2}\pi(\lfloor x \rfloor) = \tfrac{1}{2}\pi(x) > \frac{x}{2\log x}.$$

Let $f(x) = x/(2\log x)$. On $[2, 17)$ the function T equals $2, 3, 4, 5$ on the intervals $[2, 8)$, $[8, 9)$, $[9, 12)$, $[12, 17)$ respectively. Since f decreases on $(1, e]$ and increases on $[e, \infty)$, on each of these intervals f is bounded by its values at the two endpoints. These values are $f(2) < 1.45$, $f(8) < 1.93$, $f(9) < 2.05$, $f(12) < 2.42$ and $f(17) < 3.01$, each smaller than the corresponding value of T. Finally, for $1 < x < 2$ we have $2\log x < x$, so $f(x) > 1 = T(x)$. □

At $n = 7$ we have $\pi(7) = 4$ and $T(7) = 2$, the refactorable numbers up to 7 being 1 and 2. For $100 \leq n \leq 10^8$ the ratio $2T(n)/\pi(n)$ is smallest at $n = 439$, where it equals $2 \cdot 44/85 = 88/85 \approx 1.0353$ (Section 7.1).

1.3 The method

The proof does not use Zelinsky's bound. It starts from an elementary observation (Lemma 3.1):

> *If $k < 2^{52}$, then every prime factor of $\tau(k)$ is at most 47.*

Indeed, every exponent e in the factorization of such a k satisfies $e \leq 51$, so every factor $e + 1$ of $\tau(k)$ is at most $52 < 53$. Write $k = s\,r$, where s is the 47-smooth part and r the 47-rough part of k. Then $\tau(k)$ is coprime to r, and therefore

$$\tau(k) \mid k \iff \tau(s)\,\tau(r) \mid s.$$

So the rough part matters only through its exponent pattern. When r is squarefree, only the number $\omega(r)$ of its prime factors matters. We call this the *smooth–rough τ-decomposition*. It turns a divisibility condition on k into a counting problem for almost-primes, which reduces to prime counting. We use it in three ways.

(A) **Exact formula** (Theorem 3.3). For $x < 2^{52}$, $T(x)$ is an explicit finite combination of counting functions Φ_j^Q of squarefree 47-rough integers with j prime factors. By Lemma 3.4 these reduce to values of π. Combinatorial prime-counting algorithms [2, 4, 9] then compute $T(10^{14})$ in under three minutes.

(B) **Certified checkpoint chains** (Section 4). The decomposition also gives, for *all* x, a lower bound $L(x) \le T(x)$ (Proposition 3.6). Since π and T are nondecreasing, a single inequality $\pi(b) < 2L(a)$ proves $\pi(n) < 2T(n)$ for *every* $n \in [a, b]$ (Lemma 4.1). Twenty-seven such links cover $[10^{11}, 10^{14}]$.

(C) **Template tail** (Theorem 5.1). A *template* is an integer $m = s\,p_1 \cdots p_{j-1}$ such that mq is refactorable for every prime $q > p_{j-1}$. A finite family of about $2 \cdot 10^7$ templates, combined with the Rosser–Schoenfeld inequalities [7], gives

$$T(x) > 0.53934 \frac{x}{\log x} - 2.41 \cdot 10^{10} \qquad (x \ge 10^{14}).$$

This implies $\pi(x) < 2T(x)$ for every real $x \ge 10^{14}$.

The work follows the scheme *new problem* → *new method* → *application*. The new problem is the following.

> **Prime-count reduction problem for τ-divisibility.** Can the counting function $T(x)$, defined by a divisor-function condition, be computed exactly, and bounded below with certificates, using only values of the prime-counting function? On which range of x?

Theorem 3.3 solves the exact version for all $x < 2^{52}$; the threshold 2^{52} is sharp for the underlying smoothness property (Lemma 3.1). Proposition 3.6 and Lemma 3.7 solve the lower-bound version for all x. Theorem 1.2 is the application to a known open problem. We are not aware of an earlier use of this reduction to compute or bound $T(x)$.

1.4 New numerical data

Theorem 3.3 yields three new terms of OEIS A111291, the number of refactorable numbers $\le 10^n$:

$$a(12) = 26\,568\,322\,474, \qquad a(13) = 251\,342\,379\,436, \qquad a(14) = 2\,388\,671\,712\,980.$$

Each value was recomputed with every applicable smoothness bound $B \in \{37, 43, 47, 53\}$, and all runs agree (Section 7.3). The same program reproduces all previously known terms $a(0), \ldots, a(11)$.

1.5 Computer assistance and verifiability

The proof is computer-assisted. It uses three computations:

(a) an exhaustive sieve up to 10^{11};

(b) 85 evaluations each of L and π along two chains;

(c) one finite sum over about $2 \cdot 10^7$ templates.

Every number used by a chain is printed in Appendix A, and the complete source code is in Appendix B. Section 7.4 lists the independent cross-checks, among them:

- a second prime-counting algorithm for all values of π used by the chains;

- a brute-force recount of L;
- two independent implementations of the tail certificate;
- agreement with the twelve previously known terms $a(0), \dots, a(11)$ of A111291.

2 Notation and classical inputs

Throughout, p, q denote primes, log is the natural logarithm, $\omega(k)$ is the number of distinct prime factors of k, and $v_p(k)$ is the exponent of p in k. A positive integer is *B-smooth* if all its prime factors are $\le B$, and *B-rough* if all of them are $> B$; the integer 1 is both. For $k \ge 1$ put

$$s(k) = \prod_{p \le 47} p^{v_p(k)}, \qquad r(k) = \frac{k}{s(k)},$$

the 47-smooth part and the 47-rough part of k. Note that $\pi(47) = 15$ and that 53 is the least prime exceeding 47. The functions π and T are nondecreasing on $[0, \infty)$.

The only analytic inputs are two classical explicit bounds.

Lemma 2.1 (Rosser–Schoenfeld [7]). *(a)* $\pi(x) > x/\log x$ *for all* $x \ge 17$ [7, Corollary 1, (3.5)].

(b) $\pi(x) < \dfrac{x}{\log x}\left(1 + \dfrac{3}{2\log x}\right)$ *for all* $x > 1$ [7, Theorem 1, (3.2)].

As a sanity check, both inequalities were also confirmed numerically at all 87 distinct arguments of π that occur in the chains of Section 4.

3 The smooth–rough τ-decomposition

Lemma 3.1 (τ-smoothness threshold). *Let P be a prime and k a positive integer with $k < 2^{P-1}$. Then every prime factor of $\tau(k)$ is smaller than P. The bound is sharp, since $\tau(2^{P-1}) = P$. In particular, if $k < 2^{52}$ then $\tau(k)$ is 47-smooth.*

Proof. We have $\tau(k) = \prod_{p \mid k}(v_p(k) + 1)$. For each prime $p \mid k$, $2^{v_p(k)} \le p^{v_p(k)} \le k < 2^{P-1}$, hence $v_p(k) + 1 \le P - 1$. Every prime factor of an integer in $[1, P-1]$ is smaller than P. For $P = 53$, the primes smaller than P are exactly the primes ≤ 47. □

Corollary 3.2. *Let $1 \le k < 2^{52}$, $s = s(k)$ and $r = r(k)$. Then $\tau(k) \mid k$ if and only if $\tau(s)\,\tau(r) \mid s$.*

Proof. Since $\gcd(s, r) = 1$, we have $\tau(k) = \tau(s)\tau(r)$. By Lemma 3.1, $\tau(k)$ is 47-smooth and hence coprime to the 47-rough integer r. Therefore $\tau(k) \mid sr$ if and only if $\tau(k) \mid s$. □

We split the rough part further: $r = u\,v$ with

$$u = \prod_{q \mid r,\ v_q(r) \ge 2} q^{v_q(r)}, \qquad v = \prod_{q \mid r,\ v_q(r) = 1} q.$$

Thus u is powerful, v is squarefree, $\gcd(u, v) = 1$, and $\tau(r) = \tau(u)\,2^{\omega(v)}$. We use the following notation.

- $\mathcal{S} = \{s \ge 1 : \ s \text{ is 47-smooth and } \tau(s) \mid s\}$. For $s \in \mathcal{S}$ put $d(s) = s/\tau(s)$ and $J(s) = v_2(d(s))$.
- $\mathcal{U}$ is the set of 47-rough powerful integers, including 1. For $u \in \mathcal{U}$, $Q(u)$ is the set of prime factors of u.

- For a finite set Q of primes > 47, an integer $j \geq 0$ and a real $y \geq 0$,

$$\Phi_j^Q(y) = \#\Big\{v \leq y : \ v \text{ squarefree and 47-rough}, \ \omega(v) = j, \ \gcd\Big(v, \prod_{q \in Q} q\Big) = 1\Big\},$$

and $\Phi_j = \Phi_j^{\emptyset}$.

Theorem 3.3 (Exact formula)**.** *For every real x with $1 \leq x < 2^{52}$,*

$$T(x) = \sum_{s \in \mathcal{S}} \sum_{\substack{u \in \mathcal{U} \\ \tau(u) \mid d(s)}} \sum_{j=0}^{v_2(d(s)/\tau(u))} \Phi_j^{Q(u)}\Big(\frac{x}{su}\Big), \tag{1}$$

where only the finitely many pairs (s, u) with $su \leq x$ contribute.

Proof. Every integer $k \in [1, x]$ factors uniquely as $k = s\,u\,v$ with $s = s(k)$ 47-smooth, $u \in \mathcal{U}$, and v squarefree, 47-rough and coprime to u. Conversely, each such triple with $suv \leq x$ comes from exactly one k. By Corollary 3.2, k is refactorable if and only if $\tau(s)\tau(u)2^{\omega(v)} \mid s$. This holds if and only if

$$\tau(s) \mid s, \qquad \tau(u) \mid d(s), \qquad 2^{\omega(v)} \mid d(s)/\tau(u),$$

that is, if and only if $s \in \mathcal{S}$, $\tau(u) \mid d(s)$ and $\omega(v) \leq v_2(d(s)/\tau(u))$. For fixed (s, u), the number of admissible v with $\omega(v) = j$ is $\Phi_j^{Q(u)}(x/(su))$. □

Lemma 3.4 (Reduction to prime counts)**.** *Let $y \geq 0$ and let Q be a finite set of primes > 47. For a prime $p \geq 47$, let $\Phi_j^{(>p)}$ be defined like Φ_j but counting only integers all of whose prime factors exceed p; thus $\Phi_j = \Phi_j^{(>47)}$.*

(i) $\Phi_0^Q(y) = 1$ *if $y \geq 1$ and 0 otherwise. Moreover* $\Phi_1^{(>p)}(y) = \max\{0, \pi(y) - \pi(p)\}$*; in particular* $\Phi_1(y) = \max\{0, \pi(y) - 15\}$.

(ii) For $j \geq 2$,

$$\Phi_j^{(>p)}(y) = \sum_{p' > p, \ p'^{\,j} \leq y} \Phi_{j-1}^{(>p')}(y/p'),$$

where p' runs over primes.

(iii) For a prime $q \notin Q$ with $q > 47$, and for $j \geq 1$,

$$\Phi_j^{Q \cup \{q\}}(y) = \Phi_j^Q(y) - \Phi_{j-1}^{Q \cup \{q\}}(y/q).$$

Proof. Part (i) is immediate. For (ii), classify v by its least prime factor p'. Then $v = p'v'$, where $v' \leq y/p'$ has $j - 1$ prime factors, all larger than p'. Since $v' > p'^{\,j-1}$, such v exist only if $p'^{\,j} < y$. For (iii), the integers counted by $\Phi_j^Q(y)$ but divisible by q are exactly $v = qv'$, where $v' \leq y/q$ is coprime to $Q \cup \{q\}$ and $\omega(v') = j - 1$. □

Hence (1) expresses $T(x)$ entirely through finitely many values of π: values at points x/n with n a positive integer, and values $\pi(p)$ at primes $p \leq \sqrt{x}$. The largest argument needed is $x/8$, because $s = 8$ is the smallest element of $\mathcal{S}$ with $J(s) \geq 1$.

Remark 3.5 (Cost)**.** The work is dominated by the evaluations of $\Phi_2(y)$ for $y = x/(su)$, each of which needs $O(\sqrt{y})$ look-ups of π. The series $\sum_{s \in \mathcal{S}} s^{-1/2}$ converges quickly because $\mathcal{S}$ consists of 47-smooth numbers. So, heuristically, the number of look-ups is $x^{1/2+o(1)}$. To this one adds the cost of the prime-counting oracle: a sieve table up to some bound Z_0, and the combinatorial algorithms of Deléglise–Rivat and Gourdon [2, 4] for the few arguments above Z_0. The proof needs no complexity statement; measured running times are in Table 3.

Proposition 3.6 (Universal lower bound)**.** *For every real $x \geq 1$, with no upper restriction on x,*

$$T(x) \;\geq\; L(x) := \sum_{s \in \mathcal{S}} \sum_{j=0}^{J(s)} \Phi_j\Big(\frac{x}{s}\Big).$$

For $x < 2^{52}$, $L(x)$ is exactly the number of refactorable $k \leq x$ whose 47-rough part is squarefree; these are the terms with $u = 1$ in (1).

Proof. Let $s \in \mathcal{S}$, $0 \leq j \leq J(s)$, and let $v \leq x/s$ be squarefree and 47-rough with $\omega(v) = j$. Put $k = sv$. Then $\tau(k) = \tau(s)\, 2^j$. Since $2^j \mid d(s)$, the number $\tau(s)2^j$ divides $\tau(s)d(s) = s$, which divides k. Moreover $(s, v) = (s(k), r(k))$, so distinct pairs give distinct k. This argument does not use Lemma 3.1. The last statement is Theorem 3.3 restricted to $u = 1$. □

The bound is very tight. At $x = 10^{12}$ we have $L(x) = 26\,549\,555\,866$ and $T(x) = 26\,568\,322\,474$, so $L/T = 0.99929$.

Lemma 3.7 (Templates)**.** *A* template *consists of:*

- *an integer $j \geq 1$;*
- *an element $s \in \mathcal{S}$ with $J(s) \geq j$, i.e. with $2^j \tau(s) \mid s$;*
- *primes $53 \leq p_1 < \cdots < p_{j-1}$.*

Its value *is $m = s\, p_1 \cdots p_{j-1}$. Put $p(m) = p_{j-1}$, and $p(m) = 47$ if $j = 1$. The value determines the template, since $s = s(m)$ and the p_i are the prime factors of $r(m)$; we therefore identify a template with its value m.*

For every template m and every prime $q > p(m)$, the integer mq is refactorable, and distinct pairs (m, q) give distinct integers. Consequently, for every finite set $\mathcal{T}$ of templates and every real $x > 0$,

$$T(x) \;\geq\; \sum_{m \in \mathcal{T}} \Big(\pi\Big(\frac{x}{m}\Big) - \pi(p(m))\Big). \tag{2}$$

Proof. We have $mq = s\, v$ with $v = p_1 \cdots p_{j-1} q$ squarefree, 47-rough and $\omega(v) = j \leq J(s)$. So mq is refactorable by the argument in the proof of Proposition 3.6. From $k = mq$ we recover $s = s(k)$ and $v = r(k)$; then q is the largest prime factor of v, and $p_1, \ldots, p_{j-1}$ are the others. For fixed m, the number of admissible primes $q \leq x/m$ is

$$\max\{0,\; \pi(x/m) - \pi(p(m))\} \;\geq\; \pi(x/m) - \pi(p(m)).$$

□

4 Monotone checkpoints and certified chains

Lemma 4.1 (Monotone checkpoint principle)**.** *Let $1 \leq a \leq b$ be real numbers with $\pi(b) \leq 2L(a)$. Then $\pi(n) \leq 2T(n)$ for every real $n \in [a, b]$. If $\pi(b) < 2L(a)$, the inequality is strict on $[a, b]$.*

Proof. For $n \in [a, b]$, monotonicity and Proposition 3.6 give $\pi(n) \leq \pi(b) \leq 2L(a) \leq 2T(a) \leq 2T(n)$. □

Definition 4.2. A *certified chain* from a_0 to a_N is a sequence of integers $a_0 < a_1 < \cdots < a_N$ with $\pi(a_{i+1}) < 2L(a_i)$ for $0 \leq i < N$. By Lemma 4.1, it proves $\pi(n) < 2T(n)$ for all real $n \in [a_0, a_N]$.

A link certifies an interval of any length, at the cost of one evaluation of L and one of π. The ratio $2L(x)/\pi(x)$ grows slowly, from 1.135 at $x = 10^6$ to 1.49 at $x = 10^{14}$, so each link can extend the certified range by a factor up to about $2L(a)/\pi(a)$.

We build chains greedily. Given a_i, we set $\theta = 1.3$ and try $a_{i+1} = \lfloor \theta a_i \rfloor$. As long as $\pi(a_{i+1}) > 2L(a_i)$, we replace θ by $1 + 0.8(\theta - 1)$ and try again. For a target endpoint Y, the last link is truncated at $\lfloor 1.0001\, Y \rfloor + 1$.

Proposition 4.3. *The sequences in Tables 4 and 5 (Appendix A) are certified chains:*

- C_1 *from* 10^6 *to* 100 010 000 001, *with* 58 *links;*
- C_2 *from* 10^{11} *to* 100 010 000 000 001, *with* 27 *links.*

Consequently $\pi(n) < 2T(n)$ *for every real* $n \in [10^6,\ 1.0001 \cdot 10^{14}]$.

The smallest absolute margin $2L(a_{i-1}) - \pi(a_i)$ in C_1 is 54 (row 4 of Table 4). In C_2 the relative margin $(2L(a_{i-1}) - \pi(a_i))/\pi(a_i)$ is at least 0.0656 for every link.

5 The tail $x \geq 10^{14}$

Fix $X = 10^{14}$ and $A = X/\log X = 3.102103\ldots \times 10^{12}$. Let $\mathcal{T}_X$ be the set of templates m with $j \leq 4$ that satisfy

$$\frac{A}{m} > \pi(p(m)). \tag{3}$$

This selection rule is a choice. Any finite set of templates with $X/m \geq 17$ would be admissible in the argument below. The rule keeps a template exactly when, at $x = X$, its main term A/m exceeds its error term $\pi(p(m))$. Put

$$W = \sum_{m \in \mathcal{T}_X} \frac{1}{m}, \qquad E = \sum_{m \in \mathcal{T}_X} \pi(p(m)).$$

Table 1 lists the contributions by the number j of rough prime factors of mq.

Table 1: The template family $\mathcal{T}_X$ for $X = 10^{14}$.

j	number of templates	$W_j = \sum 1/m$	$E_j = \sum \pi(p(m))$
1	280 236	0.382026077834	4 203 540
2	4 886 885	0.135723931453	16 394 494 868
3	12 213 645	0.021055159405	7 227 443 076
4	2 784 611	0.000539288928	402 044 629
total	20 165 377	0.539344457620	24 028 186 113

Theorem 5.1 (Template tail)**.** *For every real* $x \geq 10^{14}$,

$$T(x) > W \frac{x}{\log x} - E, \qquad W = 0.539344457\ldots, \quad E = 24\,028\,186\,113,$$

and consequently $\pi(x) < 2T(x)$.

Proof. Let $x \geq X$ and $m \in \mathcal{T}_X$. By (3), $x/m \geq X/m > \pi(p(m)) \log X \geq 15 \log X > 17$. Lemma 2.1(a) gives

$$\pi(x/m) > \frac{x/m}{\log(x/m)} \geq \frac{x}{m \log x},$$

and summing (2) over $\mathcal{T}_X$ yields $T(x) > Wx/\log x - E$. By Lemma 2.1(b) and $\log x \geq \log X$,

$$\pi(x) < \frac{x}{\log x}\Big(1 + \frac{3}{2\log X}\Big).$$

Therefore

$$2T(x) - \pi(x) > \frac{x}{\log x}\, D - 2E, \qquad D := 2W - 1 - \frac{3}{2\log X} = 0.0321573636\ldots > 0.$$

Since $x/\log x$ is increasing for $x \geq e$, the right-hand side is at least $A\,D - 2E = 5.1699 \cdot 10^{10} > 0$. □

Corollary 5.2. *$T(x) > 0.53\,x/\log x$ for every real $x \geq 10^{14}$.*

Proof. By Theorem 5.1, $T(x) - 0.53\,x/\log x > (W - 0.53)\,x/\log x - E \geq (W - 0.53)A - E > 2.89 \cdot 10^{10} - 2.41 \cdot 10^{10} > 0$. □

Numerical rigor. The integer E is computed exactly. The number W is a sum of 20 165 377 positive terms. It was computed by two independent programs (Appendices B.7 and B.8):

- one in 64-bit-mantissa extended precision with sequential summation;
- one in IEEE double precision with prefix sums over the primes and a different loop structure.

The two values agree to all twelve printed decimals. An a priori bound for the rounding error of either computation is below 10^{-9} in absolute value. The slack in the final inequality is $D - 2E/A = 0.01667\ldots$, more than seven orders of magnitude larger.

The selection rule (3) is evaluated in floating point, so a borderline template might be misclassified. This cannot affect validity: the proof uses only $X/m > 17$, and every selected template satisfies $X/m > \pi(p(m))\log X > 480$.

Remark 5.3 (Why templates with several rough primes are needed)**.** The templates with $j = 1$ contribute only $2W_1 = 0.7641$, and no family of single-prime templates can do much better. The partial sum of $\sum_{s\in\mathcal{S},\,J(s)\geq 1} 1/s$ over $s \leq 10^{16}$ is 0.3820268. By Rankin's trick, the remainder is at most $10^{-8}\prod_{p\leq 47}(1 - p^{-1/2})^{-1} < 3 \cdot 10^{-6}$. So every family of templates with $j = 1$ has $2W < 0.7641$, whereas the argument of Theorem 5.1 needs $2W > 1$. Templates with two and three rough primes supply the missing density. Their total weight grows only slowly with X (roughly like $\log\log X$), which is why the argument starts to work only near 10^{14}. For instance, at $X = 10^{12}$ the same construction gives $2W = 1.0077$, which is less than $1 + 3/(2\log X) = 1.0543$.

6 Proof of Theorem 1.2

The interval $[1, \infty)$ is covered by four overlapping pieces.

(i) $1 \leq n \leq 10^{11}$: exhaustive computation (Section 7.1). Over this range the minimum of $2T(n) - \pi(n)$ is 0, and it is first attained at $n = 7$. An independent sequential program shows that, for $1 \leq n \leq 10^8$, equality $\pi(n) = 2T(n)$ holds for exactly one n, namely $n = 7$.

(ii) $10^6 \leq n \leq 1.0001 \cdot 10^{11}$: chain C_1 (Proposition 4.3); the inequality is strict.

(iii) $10^{11} \leq n \leq 1.0001 \cdot 10^{14}$: chain C_2 (Proposition 4.3); strict.

(iv) $n \geq 10^{14}$: Theorem 5.1; strict.

Together these give $\pi(n) \le 2T(n)$ for all $n \ge 1$. By (ii)–(iv) the inequality is strict for $n \ge 10^6$, and by (i) the only equality below 10^8 occurs at $n = 7$. This proves Theorem 1.2. Piece (ii) is not needed for the inequality itself, which already follows from (i), (iii) and (iv). It is used only for the equality statement between 10^8 and 10^{11}, and it also gives a second, independent proof of the range $[10^6, 10^{11}]$. □

7 Computations

All computations ran on a shared virtual machine with four cores of an Intel Xeon processor at 2.10 GHz and 15 GB of memory. The software was GCC 13.3, Python 3.13, primecount 7.10 and primesieve 12.0 [9]. The code is listed in Appendix B.

7.1 Exhaustive verification up to 10^{11}

Program `psieve.c` (Appendix B.1) processes $[1, N]$ in chunks in parallel. In each chunk it computes $\tau(k)$ and primality for every k by a segmented sieve: for each prime $p \le \sqrt{N}$ it divides p out of all multiples of p in the chunk and records the exponent. It records, per chunk:

- the increments of T and π;
- the minimum of the local partial sums of $2 \cdot \mathbf{1}[\tau(k) \mid k] - \mathbf{1}[k \text{ prime}]$.

Combining the chunks gives the exact minimum of $2T(n) - \pi(n)$ over $1 \le n \le N$. For $N = 10^{11}$ this minimum is 0, first attained at $n = 7$; the run took about 41 minutes. The intermediate values $T(10^k)$ and $\pi(10^k)$ ($k \le 11$) agree with A111291 and with the classical values of $\pi(10^k)$.

The sequential program `sieve.c` (Appendix B.2) checks $\pi(n) \le 2T(n)$ for every $n \le 10^8$ and counts the equality cases. There is exactly one, $n = 7$. Over $100 \le n \le 10^8$ the minimum of $2T(n)/\pi(n)$ is $88/85$, attained at $n = 439$. A third program, `check2.c` (Appendix B.3), computes τ by a linear sieve and reproduces $T(10^8) = 3\,558\,181$ and $\pi(10^8) = 5\,761\,455$.

7.2 Evaluation of L and of the chains

Program `checkpoints.cpp` (Appendix B.5) builds a table of π up to Z_0. The table is a bit array over odd numbers with cumulative counts per 64-bit word; $Z_0 = 2 \cdot 10^{10}$ for C_2 and $2 \cdot 10^9$ for C_1. Larger arguments are passed to the primecount library [9], which by default uses Gourdon's algorithm [4].

The program enumerates $\mathcal{S}$ by depth-first search over the exponent vectors of the 15 primes ≤ 47. It evaluates $L(x)$ by Lemma 3.4, using the fact that $\Phi_{j+1}(y) = 0$ whenever $\Phi_j(y) = 0$. It then runs the greedy chain construction of Section 4. Every comparison $\pi(a_{i+1}) \le 2L(a_i)$ is exact integer arithmetic.

7.3 Exact values of T

Program `exactT_B.cpp` (Appendix B.6) implements Theorem 3.3 for a general smoothness bound B. With P the least prime exceeding B, Lemma 3.1 makes the formula valid for $x < 2^{P-1}$; the program refuses larger x. Different values of B lead to different decompositions $k = s\,u\,v$, with different sets $\mathcal{S}$ and $\mathcal{U}$ and different almost-prime counts. Agreement across several B is therefore a strong consistency check.

The program reproduces every known term $a(0), \dots, a(11)$ of A111291 ($B = 47$). It also gives the following values, which agree for every bound B that applies:

- $T(10^{12})$ for $B = 37, 43, 47, 53$;
- $T(10^{13})$ for $B = 43, 47, 53$;

- $T(10^{14})$ for $B = 47, 53$.

Table 2: $T(10^k)$, $\pi(10^k)$ and the ratio $2T/\pi$. Entries for $k \le 11$ agree with OEIS A111291; those for $k = 12, 13, 14$ are new.

k	$T(10^k)$	$\pi(10^k)$	$2T(10^k)/\pi(10^k)$
1	4	4	2.000000
2	16	25	1.280000
3	92	168	1.095238
4	665	1 229	1.082181
5	5 257	9 592	1.096122
6	44 705	78 498	1.139010
7	394 240	664 579	1.186435
8	3 558 181	5 761 455	1.235168
9	32 608 999	50 847 534	1.282619
10	302 172 507	455 052 511	1.328078
11	2 823 898 245	4 118 054 813	1.371472
12	26 568 322 474*	37 607 912 018	1.412911
13	251 342 379 436*	346 065 536 839	1.452571
14	2 388 671 712 980*	3 204 941 750 802	1.490618

7.4 Cross-checks

1. *Prime counts.* All 85 values $\pi(a_{i+1})$ used by the chains were recomputed with the primecount command-line tool using the Deléglise–Rivat algorithm (`-deleglise-rivat`). This is a different algorithm from the library default, and all 85 values agree.

2. *The lower bound L.* Program `bruteL.c` (Appendix B.4) factors every $k \le N$ and tests the defining condition of L directly. It gives $L(10^6) = 44\,559$ and $L(10^8) = 3\,553\,787$, equal to the values computed through Lemma 3.4.

3. *Exact counts.* Besides the agreement with A111291 for $k \le 11$ and with the exhaustive sieve for $k \le 11$, `exactT_B` agrees with the sieve at the non-round points $4, 8, 12, 16, 20 \times 10^7$, and across the smoothness bounds listed in Section 7.3. Consistency $L(x) \le T(x)$ holds at $x = 10^{12}$.

4. *Tail certificate.* The C++ program `tail.cpp` and the independent Python program `tail_check.py` agree on every W_j to twelve decimals, and on every template count and E_j exactly. At $X = 10^{12}$ both report failure with identical numbers.

5. *Reproducibility.* Both chains were recomputed from scratch, and the outputs are identical to those listed in Appendix A.

7.5 Timings

The contrast between the exhaustive sieve and the decomposition is the point of the method. Sieving to 10^{11} takes about 41 minutes on four cores. The exact formula gives $T(10^{14})$, a value 1000 times further out, in under three minutes, and the whole certificate for $[10^{11}, 10^{14}]$ takes a few minutes.

Table 3: Wall-clock times on the machine described above (shared with other jobs).

task	program	time
exhaustive check, $n \le 10^8$ (sequential)	`sieve.c`	7.5 s
τ by linear sieve, $n \le 10^8$	`check2.c`	4.3 s
exhaustive check, $n \le 10^9$ (4 threads)	`psieve.c`	16 s
exhaustive check, $n \le 10^{11}$ (4 threads)	`psieve.c`	about 41 min
chain C_1 ($10^6 \to 1.0001 \cdot 10^{11}$)	`checkpoints.cpp`	12 s
chain C_2 ($10^{11} \to 1.0001 \cdot 10^{14}$)	`checkpoints.cpp`	157 s
$T(10^k)$, $1 \le k \le 11$	`exactT_B.cpp`	11 s
$T(10^{12}), T(10^{13}), T(10^{14})$	`exactT_B.cpp`	153 s
tail certificate (C++)	`tail.cpp`	1.2 s
tail certificate (Python)	`tail_check.py`	5.7 s

8 Discussion

What is new. To our knowledge the following four ingredients are new:

- the τ-smoothness threshold (Lemma 3.1) used as a reduction device;
- the exact prime-count formula (1) for $T(x)$;
- the use of the universal lower bound L inside monotone certified chains;
- the template tail with an explicit finite certificate.

Together they make Colton's conjecture a finite, checkable statement.

Generality. The same reasoning applies whenever a condition $f(k) \mid k$ involves a function f that depends only on the exponent pattern of k. Examples are $f = \tau^a$, $f(k) = 2^{\omega(k)}$, $f(k) = \prod_{p \mid k}(v_p(k) + c)$, and the higher divisor functions τ_m. Below an explicit threshold, $f(k)$ is smooth with respect to a bound depending only on $\log k$. The counting function of $\{k : f(k) \mid k\}$ then reduces to prime counts, and certified lower bounds and chains apply verbatim. We expect this to settle other explicit inequalities about such sets with modest computation.

Trust base. The proof relies on:

- the two classical inequalities of Lemma 2.1;
- the correctness of the compiled programs and of the primecount library.

The cross-checks of Section 7.4 are designed so that an error in any single component would very likely have been detected. In particular, two prime-counting algorithms agree on every π-value used by the chains. The certificates are small: two chains of 85 links in total, and one inequality with an integer E and a sum W. A formal verification in Lean would need formal versions of Lemma 2.1 together with either a verified prime-counting procedure or the exported certificates; we leave this for future work.

Relation to Zelinsky's bound. Our tail threshold 10^{14} is of the same order as Zelinsky's $7.42 \cdot 10^{13}$ [10]. We could not consult the full text of [10] (only its abstract and the summary in [3]). Since our argument does not use it, Theorem 1.2 is independent of the details of that paper. The computations of Sections 4–5 confirm, independently, that no counterexample exists above $7.42 \cdot 10^{13}$.

A The certified chains

Row i describes the link from a_{i-1} to a_i. It lists the left end a_{i-1}, the certified lower bound $L(a_{i-1}) \le T(a_{i-1})$, the right end a_i, the prime count $\pi(a_i)$, and the margin $2L(a_{i-1}) - \pi(a_i) > 0$. By Lemma 4.1, row i proves $\pi(n) < 2T(n)$ for all $n \in [a_{i-1}, a_i]$.

Table 4: Chain C_1 from 10^6 to 100 010 000 001 (58 links).

i	a_{i-1}	$L(a_{i-1})$	a_i	$\pi(a_i)$	$2L(a_{i-1}) - \pi(a_i)$
1	1000000	44559	1122879	87370	1748
2	1122879	49693	1260858	97245	2141
3	1260858	55414	1415792	108202	2626
4	1415792	61770	1633257	123486	54
5	1633257	70678	1884125	140929	427
6	1884125	80888	2173526	160879	897
7	2173526	92540	2507379	183575	1505
8	2507379	105925	2892512	209610	2240
9	2892512	121267	3336801	239337	3197
10	3336801	138793	3849333	273277	4309
11	3849333	158938	4440590	312035	5841
12	4440590	181997	5122664	356422	7572
13	5122664	208350	5909505	407043	9657
14	5909505	238650	6817204	464971	12329
15	6817204	273328	7864326	531252	15404
16	7864326	313053	9374276	625813	293
17	9374276	370005	11174136	737269	2741
18	11174136	437346	13319570	868703	5989
19	13319570	516916	15876927	1023714	10118
20	15876927	611168	18925296	1206545	15791
21	18925296	722563	22558952	1422165	22961
22	22558952	854507	26890270	1676623	32391
23	26890270	1010556	32053201	1976861	44251
24	32053201	1195187	38207415	2330927	59447
25	38207415	1413832	45543238	2749168	78496
26	45543238	1672411	54287539	3242277	102545
27	54287539	1978644	64710746	3824526	132762
28	64710746	2341370	80241325	4682610	130
29	80241325	2877255	99499242	5734218	20292
30	99499242	3536779	123379060	7023033	50525
31	123379060	4347348	152990034	8602884	91812
32	152990034	5345211	189707642	10540048	150374
33	189707642	6572032	235237476	12914350	229714
34	235237476	8082080	291694470	15826422	337738
35	291694470	9939702	361701142	19397498	481906
36	361701142	12226220	448509416	23778210	674230
37	448509416	15039304	556151675	29151098	927510
38	556151675	18503095	689628076	35743400	1262790
39	689628076	22765533	855138814	43831785	1699281
40	855138814	28013539	1060372129	53756452	2270626
41	1060372129	34474062	1314861439	65936822	3011302
42	1314861439	42429028	1709319870	84601781	256275
43	1709319870	54661887	2222115831	108567743	756031
44	2222115831	70433180	2888750580	139347766	1518594
45	2888750580	90763668	3755375754	178882751	2644585
46	3755375754	116978945	4881988480	229668180	4289710
47	4881988480	150783241	6346585024	294919873	6646609
48	6346585024	194377235	8250560531	378765377	9989093
49	8250560531	250608026	10725728690	486519277	14696775
50	10725728690	323137367	13943447297	625019759	21254975
51	13943447297	416703453	18126481486	803057904	30349002
52	18126481486	537420518	23564425931	1031954908	42886128
53	23564425931	693175813	30633753710	1326266604	60085022
54	30633753710	894165052	39823879823	1704739831	83590273
55	39823879823	1153536085	51771043769	2191496567	115575603
56	51771043769	1488288985	67302356899	2817581748	158996222
57	67302356899	1920366218	87493063968	3622985305	217747131
58	87493063968	2478098104	100010000001	4118449214	837746994

Table 5: Chain C_2 from 10^{11} to 100 010 000 000 001 (27 links).

i	a_{i-1}	$L(a_{i-1})$	a_i	$\pi(a_i)$	$2L(a_{i-1}) - \pi(a_i)$
1	100000000000	2821796492	130000000000	5296138250	347454734
2	130000000000	3641815796	169000000000	6812015990	471615602
3	169000000000	4700523176	219700000000	8762780454	638265898
4	219700000000	6067511562	285610000000	11273421612	861601512
5	285610000000	7832681227	371293000000	14504962265	1160400189
6	371293000000	10112155956	482680900000	18664796707	1559515205
7	482680900000	13056023905	627485170000	24020108257	2091939553
8	627485170000	16858178972	815730721000	30915134830	2801223114
9	815730721000	21769209101	1060449937300	39793316656	3745101546
10	1060449937300	28112906109	1378584918490	51226167825	4999644393
11	1378584918490	36307746289	1792160394037	65950014530	6665478048
12	1792160394037	46894640770	2329808512248	84913875301	8875406239
13	2329808512248	60572588584	3028751065922	109340886366	11804290802
14	3028751065922	78245226849	3937376385698	140807456610	15682997088
15	3937376385698	101080639246	5118589301407	181345707262	20815571230
16	5118589301407	130588628609	6654166091829	233575103883	27602153335
17	6654166091829	168721318806	8650415919377	300872720222	36569917390
18	8650415919377	218002205107	11245540695190	387592707670	48411702544
19	11245540695190	281694244063	14619202903747	499348744307	64039743819
20	14619202903747	364016019296	19004963774871	643379961539	84652077053
21	19004963774871	470422282261	24706452907332	829020898377	111823666145
22	24706452907332	607966834055	32118388779531	1068310022059	147623646051
23	32118388779531	785770723938	41753905413390	1376773153539	194768294337
24	41753905413390	1015629792485	54280077037407	1774435044640	256824540330
25	54280077037407	1312798803893	70564100148629	2287125394765	338472213021
26	70564100148629	1697006858491	91733330193217	2948162225971	445851491011
27	91733330193217	2193771404114	100010000000001	3205251958942	1182290849286

B Source code

The programs below are exactly those that produced the results in this paper.

The programs can be compiled as follows.

```
gcc -O3 -march=native -fopenmp -o psieve psieve.c -lm
gcc -O2 -o sieve sieve.c -lm
gcc -O2 -o check2 check2.c
gcc -O2 -o bruteL bruteL.c
g++ -O3 -march=native -o checkpoints checkpoints.cpp -lprimecount -lprimesieve
g++ -O3 -march=native -o exactT_B exactT_B.cpp -lprimecount -lprimesieve
g++ -O2 -march=native -o tail tail.cpp
```

The results in this paper were produced by the following invocations.

```
# exhaustive verification, n <= 1e11 (args: N, chunk size, output table)
./psieve 100000000000 50000000 chunks.txt
# sequential verification with equality count, n <= 1e8
./sieve 100000000
# chains (args: Z0, prime-list limit, start, end, initial step ratio)
./checkpoints 2000000000 20000000 1000000 100000000000 1.3              # C1
./checkpoints 20000000000 20000000 100000000000 100000000000000 1.3     # C2
# tail certificate (args: X, prime-list limit, max j)
./tail 1e14 50000000 4
python3 tail_check.py 1e14 20000000
# exact values (args: B, Z0, prime-list limit, x_1, x_2, ...)
./exactT_B 47 20000000000 30000000 1000000000000 10000000000000 \
           100000000000000
```

B.1 Exhaustive parallel verification (`psieve.c`)

```
// Parallel exhaustive verification of pi(n) <= 2*T(n) for all 1 <= n <= N.
// T(n) = #{k<=n : tau(k) | k}. Chunks processed in parallel; per-chunk we record
// dT, dP and the minimum of (2*T_loc - P_loc) over prefixes inside the chunk, then combine.
```

```
// Also records T and pi at checkpoints (powers of 10 and multiples of chunk).
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <math.h>
#include <omp.h>
typedef uint64_t u64; typedef int64_t i64; typedef uint32_t u32;
static u32 *pr; static u64 np;
typedef struct { u64 dT, dP; i64 minloc; u64 argmin; } chunkres;
static void do_chunk(u64 lo, u64 hi, u64 *rem, u32 *tau, chunkres *out) {
    u64 len = hi - lo + 1;
    for (u64 i = 0; i < len; i++) { rem[i] = lo + i; tau[i] = 1; }
    for (u64 t = 0; t < np; t++) {
        u64 p = pr[t]; if (p * p > hi) break;
        u64 start = ((lo + p - 1) / p) * p;
        for (u64 m = start; m <= hi; m += p) {
            u64 i = m - lo; u32 e = 0; u64 r = rem[i];
            do { r /= p; e++; } while (r % p == 0);
            rem[i] = r; tau[i] *= (e + 1);
        }
    }
    u64 T = 0, P = 0; i64 minloc = INT64_MAX; u64 argmin = lo;
    for (u64 i = 0; i < len; i++) {
        u64 k = lo + i; u32 tk = tau[i]; if (rem[i] > 1) tk *= 2;
        if (k >= 2 && tk == 2) P++;
        if (k % tk == 0) T++;
        i64 v = 2 * (i64)T - (i64)P;
        if (v < minloc) { minloc = v; argmin = k; }
    }
    out->dT = T; out->dP = P; out->minloc = minloc; out->argmin = argmin;
}
int main(int argc, char **argv) {
    u64 N = strtoull(argv[1], 0, 10);
    u64 CH = argc > 2 ? strtoull(argv[2], 0, 10) : 100000000ULL;
    u64 R = (u64)sqrtl((long double)N) + 2;
    char *isc = calloc(R + 1, 1); pr = malloc(sizeof(u32) * (R + 1)); np = 0;
    for (u64 i = 2; i <= R; i++) { if (!isc[i]) { pr[np++] = (u32)i; for (u64 j = i * i; j <= R; j += i)
     isc[j] = 1; } }
    u64 nch = (N + CH - 1) / CH;
    chunkres *res = calloc(nch, sizeof(chunkres));
    #pragma omp parallel
    {
        u64 *rem = malloc(sizeof(u64) * CH); u32 *tau = malloc(sizeof(u32) * CH);
        #pragma omp for schedule(dynamic, 1)
        for (u64 c = 0; c < nch; c++) {
            u64 lo = c * CH + 1, hi = lo + CH - 1; if (hi > N) hi = N;
            do_chunk(lo, hi, rem, tau, &res[c]);
        }
        free(rem); free(tau);
    }
    u64 T = 0, P = 0; i64 globalmin = 1LL << 62; u64 gargmin = 0;
    double minratio = 1e18; u64 argr = 0;
    for (u64 c = 0; c < nch; c++) {
        i64 base = 2 * (i64)T - (i64)P;
        i64 m = base + res[c].minloc;
        if (m < globalmin) { globalmin = m; gargmin = res[c].argmin; }
        T += res[c].dT; P += res[c].dP;
        u64 hi = (c + 1) * CH; if (hi > N) hi = N;
        double r = (P > 0) ? 2.0 * (double)T / (double)P : 1e18;
        if (r < minratio && hi >= 1000000) { minratio = r; argr = hi; }
        // print checkpoint at powers of ten
        u64 x = hi; int pow10 = 1; u64 t = x; while (t > 1) { if (t % 10) { pow10 = 0; break; } t /= 10; }
        if (pow10 || c == nch - 1) printf("x=%llu T=%llu pi=%llu 2T-pi=%lld ratio=%.6f\n", (unsigned long
     long)x, (unsigned long long)T, (unsigned long long)P, (long long)(2 * (i64)T - (i64)P), r);
    }
    printf("RESULT N=%llu: min over 1<=n<=N of (2T(n)-pi(n)) = %lld at n=%llu  %s\n", (unsigned long long)
     N, (long long)globalmin, (unsigned long long)gargmin, globalmin >= 0 ? "(conjecture holds)" : "(
     COUNTEREXAMPLE)");
    // dump chunk-end table for later use
    FILE *f = fopen(argc > 3 ? argv[3] : "chunks.txt", "w");
    T = 0; P = 0;
```

```
    for (u64 c = 0; c < nch; c++) { T += res[c].dT; P += res[c].dP; u64 hi = (c + 1) * CH; if (hi > N) hi
     = N; fprintf(f, "%llu %llu %llu\n", (unsigned long long)hi, (unsigned long long)T, (unsigned long long
     )P); }
    fclose(f);
    return 0;
}
```

B.2 Sequential verification with equality count (`sieve.c`)

```
// Exhaustive verification of Colton's conjecture pi(n) <= 2 T(n) for all n <= N,
// where T(n) = #{k <= n : tau(k) | k}. Segmented sieve computing tau(k) and primality.
// Usage: ./sieve N [segment]
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <math.h>
typedef uint64_t u64; typedef uint32_t u32;
int main(int argc, char **argv) {
    u64 N = strtoull(argv[1], 0, 10);
    u64 SEG = argc > 2 ? strtoull(argv[2], 0, 10) : (1ULL << 24);
    u64 R = (u64)sqrtl((long double)N) + 2;
    // base primes up to R
    char *isc = calloc(R + 1, 1);
    u32 *pr = malloc(sizeof(u32) * (R + 1)); u64 np = 0;
    for (u64 i = 2; i <= R; i++) { if (!isc[i]) { pr[np++] = (u32)i; for (u64 j = i * i; j <= R; j += i)
     isc[j] = 1; } }
    u64 *rem = malloc(sizeof(u64) * SEG);   // remaining cofactor
    u32 *tau = malloc(sizeof(u32) * SEG);
    u64 T = 0, P = 0; // counts
    double minratio = 1e9; u64 argmin = 0; u64 tight = 0;
    // record checkpoints
    for (u64 lo = 1; lo <= N; lo += SEG) {
        u64 hi = lo + SEG - 1; if (hi > N) hi = N;
        u64 len = hi - lo + 1;
        for (u64 i = 0; i < len; i++) { rem[i] = lo + i; tau[i] = 1; }
        for (u64 t = 0; t < np; t++) {
            u64 p = pr[t]; if (p * p > hi) break;
            u64 start = ((lo + p - 1) / p) * p;
            for (u64 m = start; m <= hi; m += p) {
                u64 i = m - lo; u32 e = 0;
                while (rem[i] % p == 0) { rem[i] /= p; e++; }
                tau[i] *= (e + 1);
            }
        }
        for (u64 i = 0; i < len; i++) {
            u64 k = lo + i;
            u32 tk = tau[i]; if (rem[i] > 1) tk *= 2;
            if (k >= 2 && tk == 2) P++;
            if (k % tk == 0) T++;
            // check pi(k) <= 2 T(k)
            if (P > 2 * T) { printf("COUNTEREXAMPLE at n=%llu: pi=%llu T=%llu\n", (unsigned long long)k, (
     unsigned long long)P, (unsigned long long)T); fflush(stdout); }
            if (P > 0) {
                double r = (2.0 * (double)T) / (double)P;
                if (k >= 100 && r < minratio) { minratio = r; argmin = k; }
                if (2 * T == P) tight++;
            }
        }
        // checkpoint output at powers of 10
    }
    printf("N=%llu pi=%llu T=%llu  min 2T/pi (n>=100) = %.6f at n=%llu ; equality cases: %llu\n",
           (unsigned long long)N, (unsigned long long)P, (unsigned long long)T, minratio, (unsigned long
     long)argmin, (unsigned long long)tight);
    return 0;
}
```

B.3 Linear-sieve cross-check of T and π (`check2.c`)

```c
// independent check: tau via linear sieve (smallest prime factor) up to N
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int main(int argc,char**argv){
  long N=atol(argv[1]);
  int *spf=calloc(N+1,sizeof(int)); unsigned char *e=calloc(N+1,1); int *tau=calloc(N+1,sizeof(int)); int *
     rest=calloc(N+1,sizeof(int));
  int *pr=malloc(sizeof(int)*(N/2+10)); long np=0;
  tau[1]=1;
  for(long i=2;i<=N;i++){
    if(!spf[i]){spf[i]=i; pr[np++]=i; e[i]=1; tau[i]=2; rest[i]=1;}
    for(long j=0;j<np;j++){ long p=pr[j]; if(p>spf[i]||p*i>N) break; long m=p*i; spf[m]=p;
      if(p==spf[i]){ e[m]=e[i]+1; rest[m]=rest[i]; tau[m]=tau[rest[i]]*(e[m]+1);} else { e[m]=1; rest[m]=i
     ; tau[m]=tau[i]*2; } }
  }
  long T=0,P=0; for(long k=1;k<=N;k++){ if(k%tau[k]==0) T++; if(tau[k]==2) P++; }
  printf("N=%ld T=%ld pi=%ld\n",N,T,P);
}
```

B.4 Brute-force evaluation of L (`bruteL.c`)

```c
// Brute-force L(N): count k <= N with k = s*r, s 47-smooth, r squarefree with all primes > 47,
// omega(r) = j and 2^j * tau(s) | s. Independent check of the structured formula for L.
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int main(int argc, char **argv) {
    uint32_t N = (uint32_t)strtoul(argv[1], 0, 10);
    uint32_t *spf = calloc((size_t)N + 1, sizeof(uint32_t));
    for (uint64_t i = 2; i <= N; i++) if (!spf[i]) for (uint64_t k = i; k <= N; k += i) if (!spf[k]) spf[k
     ] = (uint32_t)i;
    uint64_t L = 0, T = 0;
    for (uint32_t k = 1; k <= N; k++) {
        uint32_t m = k; uint64_t s = 1, taus = 1, tauk = 1; int j = 0, sqf = 1;
        while (m > 1) {
            uint32_t p = spf[m]; int e = 0; uint64_t pe = 1;
            while (m % p == 0) { m /= p; e++; pe *= p; }
            tauk *= (e + 1);
            if (p <= 47) { s *= pe; taus *= (e + 1); }
            else { if (e > 1) sqf = 0; j++; }
        }
        if (k % tauk == 0) T++;
        if (sqf && s % ((taus << j)) == 0) L++;
    }
    printf("N=%u bruteL=%llu T=%llu\n", N, (unsigned long long)L, (unsigned long long)T);
    return 0;
}
```

B.5 Lower bound L and certified chains (`checkpoints.cpp`)

```c
// Rigorous lower bound L(x) <= T(x) = #{k <= x : tau(k) | k}.
// k = s*r, s 47-smooth with tau(s) | s, r squarefree with j prime factors all > 47,
// counted only when 2^j * tau(s) | s  (then tau(k) = tau(s)*2^j divides s, hence k).
// Distinct (s, r) give distinct k by unique factorization.
// L(x) = sum_{s} sum_{j=0}^{J(s)} Phi_j(floor(x/s)),  Phi_j(y) = #{r<=y squarefree, omega(r)=j, all p>47}.
// pi(z): exact via bit table for z <= Z0, else primecount::pi.
#include <bits/stdc++.h>
#include <primecount.hpp>
using namespace std;
typedef long long ll; typedef unsigned long long ull;

static ull Z0;                   // table limit
static vector<ull> bits;         // odd-only bitset: bit i <-> number 2i+1 is prime
static vector<uint32_t> blockcnt; // prefix count of primes (odd) per 64-bit word
static vector<uint32_t> smallp;  // primes up to PL for iteration
static unordered_map<ull, ll> pcache;
```

```
void build_table(ull Z) {
    Z0 = Z;
    ull nbits = Z / 2 + 1; ull nw = nbits / 64 + 2;
    bits.assign(nw, ~0ULL);
    // bit i: number 2i+1. mark 1 (i=0) composite
    auto clr = [&](ull i) { bits[i >> 6] &= ~(1ULL << (i & 63)); };
    clr(0);
    ull r = (ull)sqrtl((long double)Z) + 1;
    vector<char> sm(r + 1, 1);
    for (ull p = 3; p <= r; p += 2) if (sm[p]) {
        for (ull q = p * p; q <= r; q += 2 * p) sm[q] = 0;
        for (ull m = p * p; m <= Z; m += 2 * p) clr(m / 2);
    }
    // clear bits beyond Z
    for (ull i = nbits; i < nw * 64; i++) clr(i);
    blockcnt.assign(nw + 1, 0);
    ull c = 0;
    for (ull w = 0; w < nw; w++) { blockcnt[w] = (uint32_t)(c & 0xffffffffULL); c += __builtin_popcountll(
     bits[w]); }
    // store high parts separately: counts up to ~1e9 fit in 32 bits for Z<=2e10? pi(2e10)~8.8e8 < 4.29e9
     OK
}
inline ll pi_table(ull z) { // z <= Z0
    if (z < 2) return 0;
    if (z < 3) return 1;
    ull i = (z - 1) / 2; // number 2i+1 <= z, largest odd <= z
    ull w = i >> 6; ull b = i & 63;
    ull mask = (b == 63) ? ~0ULL : ((1ULL << (b + 1)) - 1);
    return 1 + (ll)blockcnt[w] + __builtin_popcountll(bits[w] & mask); // +1 for prime 2
}
ll PI(ull z) {
    if (z <= Z0) return pi_table(z);
    auto it = pcache.find(z); if (it != pcache.end()) return it->second;
    ll v = primecount::pi((int64_t)z);
    pcache[z] = v; return v;
}
static const ll PI47 = 15; // pi(47)
// Phi_j(y, idx): number of squarefree r <= y, omega(r)=j, all prime factors >= smallp[idx] (> 47)
ll Phi(ull y, int j, size_t idx) {
    if (j == 0) return y >= 1 ? 1 : 0;
    if (idx >= smallp.size()) { fprintf(stderr, "prime list too short\n"); exit(1); }
    ull p0 = smallp[idx];
    if (j == 1) { ll v = PI(y) - PI(p0 - 1); return v > 0 ? v : 0; }
    ll total = 0;
    for (size_t t = idx; t < smallp.size(); t++) {
        ull p = smallp[t];
        // need p * (next j-1 primes, each > p) <= y ; lower bound check p^j <= y
        long double pj = powl((long double)p, j);
        if (pj > (long double)y) break;
        total += Phi(y / p, j - 1, t + 1);
    }
    return total;
}

ll computeL(ull x, vector<pair<ull,int>> &good) {
    ll L = 0;
    for (auto &g : good) {
        if (g.first > x) continue;
        ull y = x / g.first;
        for (int j = 0; j <= g.second; j++) {
            ll f = Phi(y, j, 0);
            if (f == 0 && j > 0) break;
            L += f;
        }
    }
    return L;
}
int main(int argc, char **argv) {
    // args: Z0 PLIMIT x0 Xend q
    ull Z = strtoull(argv[1], 0, 10);
    ull PL = strtoull(argv[2], 0, 10);
    ull x0 = strtoull(argv[3], 0, 10);
    ull Xend = strtoull(argv[4], 0, 10);
```

```
    double q = atof(argv[5]);
    build_table(Z);
    { vector<char> sv(PL + 1, 1);
      for (ull i = 2; i * i <= PL; i++) if (sv[i]) for (ull k = i * i; k <= PL; k += i) sv[k] = 0;
      for (ull i = 53; i <= PL; i++) if (sv[i]) smallp.push_back((uint32_t)i); }
    const int P[15] = {2,3,5,7,11,13,17,19,23,29,31,37,41,43,47};
    vector<pair<ull,int>> good;
    ull X = (ull)(Xend * 1.5);
    function<void(int, ull, ull)> dfs = [&](int i, ull s, ull tau) {
        if (i == 15) { if (s % tau == 0) { ull qq = s / tau; int J = 0; while (qq % 2 == 0) { qq /= 2; J++;
      } good.push_back({s, J}); } return; }
        ull v = s; int e = 0;
        while (true) { dfs(i + 1, v, tau * (e + 1)); if (v > X / P[i]) break; v *= P[i]; e++; }
    };
    dfs(0, 1, 1);
    sort(good.begin(), good.end());
    fprintf(stderr, "good s: %zu\n", good.size());
    ull x = x0; ll L = computeL(x, good);
    printf("CHECKPOINT x=%llu L=%lld pi=%lld 2L/pi=%.6f\n", x, L, PI(x), 2.0 * L / PI(x)); fflush(stdout);
    int step = 0;
    while (x < Xend) {
        // find next checkpoint: largest y = x*q' (q' <= q) with pi(y) <= 2L
        double qq = q; ull y; ll py;
        while (true) {
            y = (ull)((long double)x * qq);
            if (y > Xend * 1.0001 && x < Xend) y = (ull)(Xend * 1.0001) + 1;
            py = PI(y);
            if (py <= 2 * L) break;
            qq = 1.0 + (qq - 1.0) * 0.8;
            if (qq < 1.0001) { printf("FAILED to advance at x=%llu\n", x); return 1; }
        }
        printf("  interval [%llu, %llu]: pi(right)=%lld <= 2*L(left)=%lld  OK\n", x, y, py, 2 * L); fflush
     (stdout);
        x = y; L = computeL(x, good); step++;
        printf("CHECKPOINT x=%llu L=%lld pi=%lld 2L/pi=%.6f (step %d)\n", x, L, PI(x), 2.0 * L / PI(x),
     step); fflush(stdout);
    }
    printf("VERIFIED: pi(n) <= 2T(n) for all n in [%llu, %llu]\n", x0, x);
    return 0;
}
```

B.6 Exact formula for $T(x)$ with smoothness bound B (`exactT_B.cpp`)

```
// Exact T(x) = #{k <= x : tau(k) | k} for x < 2^47 via the smooth-rough decomposition.
// k = s * u * v with s the 47-smooth part, u the powerful part of the 47-rough part
// (primes > 47 with exponent >= 2) and v the squarefree part (primes > 47 with exponent 1).
// For k <= x < 2^47 every factor e+1 of tau(k) is <= 47 (exponents of 2..47 are <= 46,
// exponents of primes >= 53 are <= 8), so tau(k) is 47-smooth and coprime to u*v. Hence
//   tau(k) | k  <=>  tau(s) * tau(u) * 2^omega(v) | s.
// T(x) = sum_{s: tau(s)|s} sum_{u: tau(u) | d(s)} sum_{j=0}^{v2(d(s)/tau(u))} Phi_j^{(u)}(x/(s u)),
// d(s) = s / tau(s), Phi_j^{(u)}(y) = #{v <= y squarefree, primes > 47, omega(v) = j, gcd(v,u) = 1}.
#include <bits/stdc++.h>
#include <primecount.hpp>
using namespace std;
typedef long long ll; typedef unsigned long long ull;

static ull Z0;
static vector<ull> bits;
static vector<uint32_t> blockcnt;
static vector<uint32_t> smallp;
static unordered_map<ull, ll> pcache;

void build_table(ull Z) {
    Z0 = Z;
    ull nbits = Z / 2 + 1; ull nw = nbits / 64 + 2;
    bits.assign(nw, ~0ULL);
    auto clr = [&](ull i) { bits[i >> 6] &= ~(1ULL << (i & 63)); };
    clr(0);
    ull r = (ull)sqrtl((long double)Z) + 1;
    vector<char> sm(r + 1, 1);
    for (ull p = 3; p <= r; p += 2) if (sm[p]) {
```

```
        for (ull q = p * p; q <= r; q += 2 * p) sm[q] = 0;
        for (ull m = p * p; m <= Z; m += 2 * p) clr(m / 2);
    }
    for (ull i = nbits; i < nw * 64; i++) clr(i);
    blockcnt.assign(nw + 1, 0);
    ull c = 0;
    for (ull w = 0; w < nw; w++) { blockcnt[w] = (uint32_t)c; c += __builtin_popcountll(bits[w]); }
    if (c > 0xffffffffULL) { fprintf(stderr, "table too large for 32-bit counts\n"); exit(1); }
}
inline ll pi_table(ull z) {
    if (z < 2) return 0;
    if (z < 3) return 1;
    ull i = (z - 1) / 2; ull w = i >> 6; ull b = i & 63;
    ull mask = (b == 63) ? ~0ULL : ((1ULL << (b + 1)) - 1);
    return 1 + (ll)blockcnt[w] + __builtin_popcountll(bits[w] & mask);
}
ll PI(ull z) {
    if (z <= Z0) return pi_table(z);
    auto it = pcache.find(z); if (it != pcache.end()) return it->second;
    ll v = primecount::pi((int64_t)z);
    pcache[z] = v; return v;
}
// squarefree r <= y, omega(r) = j, all prime factors >= smallp[idx]
ll Phi(ull y, int j, size_t idx) {
    if (j == 0) return y >= 1 ? 1 : 0;
    if (idx >= smallp.size()) { fprintf(stderr, "prime list too short\n"); exit(1); }
    ull p0 = smallp[idx];
    if (j == 1) { ll v = PI(y) - PI(p0 - 1); return v > 0 ? v : 0; }
    ll total = 0;
    for (size_t t = idx; ; t++) {
        if (t + 1 >= smallp.size()) { fprintf(stderr, "prime list too short\n"); exit(1); }
        ull p = smallp[t];
        long double pj = powl((long double)p, j);
        if (pj > (long double)y) break;
        total += Phi(y / p, j - 1, t + 1);
    }
    return total;
}
// same, additionally coprime to the primes Q[0..nq-1]
ll PhiQ(ull y, int j, const ull *Q, int nq) {
    if (j == 0) return y >= 1 ? 1 : 0;
    if (y == 0) return 0;
    if (nq == 0) return Phi(y, j, 0);
    ull q = Q[nq - 1];
    return PhiQ(y, j, Q, nq - 1) - PhiQ(y / q, j - 1, Q, nq);
}
struct Pow { ull u; ull tau; int nq; ull Q[8]; };

int main(int argc, char **argv) {
    // args: B Z0 PL x1 x2 ...
    int B = atoi(argv[1]);
    ull Z = strtoull(argv[2], 0, 10);
    ull PL = strtoull(argv[3], 0, 10);
    vector<ull> xs; for (int a = 4; a < argc; a++) xs.push_back(strtoull(argv[a], 0, 10));
    ull X = *max_element(xs.begin(), xs.end());
    vector<int> Pv; int Pnext = 0;
    for (int c = 2; ; c++) { bool pr = true; for (int d = 2; d * d <= c; d++) if (c % d == 0) { pr = false;
      break; } if (pr) { if (c <= B) Pv.push_back(c); else { Pnext = c; break; } } }
    if (Pnext - 1 >= 64 || X >= (1ULL << (Pnext - 1))) { fprintf(stderr, "x must be < 2^%d\n", Pnext - 1);
      return 1; }
    int NP = (int)Pv.size();
    build_table(Z);
    { vector<char> sv(PL + 1, 1);
      for (ull i = 2; i * i <= PL; i++) if (sv[i]) for (ull k = i * i; k <= PL; k += i) sv[k] = 0;
      for (ull i = (ull)B + 1; i <= PL; i++) if (sv[i]) smallp.push_back((uint32_t)i); }
    // good s: 47-smooth, tau(s) | s
    vector<pair<ull,ull>> good; // (s, d = s/tau(s))
    function<void(int, ull, ull)> dfs = [&](int i, ull s, ull tau) {
        if (i == NP) { if (s % tau == 0) good.push_back({s, s / tau}); return; }
        ull v = s; int e = 0;
        while (true) { dfs(i + 1, v, tau * (e + 1)); if (v > X / Pv[i]) break; v *= Pv[i]; e++; }
    };
    dfs(0, 1, 1);
```

```
    sort(good.begin(), good.end());
    // powerful 47-rough u <= X/9 (u > 1 needs tau(u) >= 3 dividing d(s), and d(s) <= 2 for s < 9)
    ull UMAX = X / 9;
    vector<Pow> pw;
    { Pow one; one.u = 1; one.tau = 1; one.nq = 0; pw.push_back(one); }
    function<void(size_t, ull, ull, Pow &)> dfsu = [&](size_t t, ull u, ull tau, Pow &cur) {
        for (size_t i = t; ; i++) {
            if (i >= smallp.size()) { fprintf(stderr, "prime list too short for u\n"); exit(1); }
            ull q = smallp[i];
            if ((long double)u * q * q > (long double)UMAX) break;
            ull v = u * q * q; int f = 2;
            while (true) {
                Pow nx = cur; nx.Q[nx.nq++] = q; nx.u = v; nx.tau = tau * (f + 1);
                pw.push_back(nx);
                dfsu(i + 1, v, tau * (f + 1), nx);
                if ((long double)v * q > (long double)UMAX) break;
                v *= q; f++;
            }
        }
    };
    { Pow root; root.u = 1; root.tau = 1; root.nq = 0; dfsu(0, 1, 1, root); }
    sort(pw.begin(), pw.end(), [](const Pow &a, const Pow &b) { return a.u < b.u; });
    fprintf(stderr, "good s: %zu, powerful rough u: %zu\n", good.size(), pw.size());
    for (ull x : xs) {
        ll T = 0;
        for (auto &g : good) {
            ull s = g.first, d = g.second;
            if (s > x) break;
            ull xs_ = x / s;
            for (auto &w : pw) {
                if (w.u > xs_) break;
                if (d % w.tau) continue;
                ull r = d / w.tau; int J = 0; while (r % 2 == 0) { r /= 2; J++; }
                ull y = xs_ / w.u;
                for (int j = 0; j <= J; j++) {
                    ll f = PhiQ(y, j, w.Q, w.nq);
                    if (f == 0 && j > 0) break;
                    T += f;
                }
            }
        }
        printf("x=%llu T(x)=%lld pi(x)=%lld 2T-pi=%lld 2T/pi=%.6f\n", x, T, PI(x), 2 * T - PI(x), 2.0 * T
     / PI(x));
        fflush(stdout);
    }
    return 0;
}
```

B.7 Template tail certificate (`tail.cpp`)

```
// Analytic tail certificate for Colton's conjecture.
// For x >= X:  T(x) >= sum_{m in M} ( pi(x/m) - pi(p(m)) )
// where m = s * p_1 ... p_{j-1}, s 47-smooth with 2^j tau(s) | s, 53 <= p_1 < ... < p_{j-1} primes,
// p(m) = p_{j-1} (or 47 if j = 1), and the counted integers are k = m*q with q prime, p(m) < q <= x/m.
// Each such k has tau(k) = 2^j tau(s) | s | k, and k determines (s, p_1..p_{j-1}, q) uniquely.
// Using pi(y) >= y/log y (y >= 17) and log(x/m) <= log x:
//   T(x) >= (x/log x) * W - E,   W = sum 1/m,  E = sum pi(p(m)).
// Using pi(x) < (x/log x)(1 + 3/(2 log x)) (x > 1):
//   2T(x) - pi(x) > (x/log x)(2W - 1 - 3/(2 log X)) - 2E, increasing in x once 2W - 1 - 3/(2 log X) > 0.
// So it suffices that (X/log X)(2W - 1 - 3/(2 log X)) >= 2E.
// m is included iff A/m > pi(p(m)) with A = X/log X (any choice is valid; this one is near-optimal),
// which also guarantees x/m >= X/m > 15 log X >= 17 for every included m.
#include <bits/stdc++.h>
using namespace std;
typedef unsigned long long ull; typedef long double ld;

int main(int argc, char **argv) {
    ld X = strtold(argv[1], 0);
    ull PMAX = argc > 2 ? strtoull(argv[2], 0, 10) : 100000000ULL;
    int JMAX = argc > 3 ? atoi(argv[3]) : 5;
    ld lX = logl(X), A = X / lX;
```

```
// primes >= 53 up to PMAX
vector<char> sv(PMAX + 1, 1); sv[0] = sv[1] = 0;
for (ull i = 2; i * i <= PMAX; i++) if (sv[i]) for (ull k = i * i; k <= PMAX; k += i) sv[k] = 0;
vector<ull> q; for (ull i = 53; i <= PMAX; i++) if (sv[i]) q.push_back(i);
size_t NQ = q.size();
vector<ld> inv(NQ + 1, 0); vector<ull> pis(NQ + 1, 0);
for (size_t t = 0; t < NQ; t++) { inv[t + 1] = inv[t] + 1.0L / (ld)q[t]; pis[t + 1] = pis[t] + (ull)(t
  + 16); }
// pi(q[t]) = t + 16 since pi(53) = 16
auto first_fail = [&](ld B, size_t lo) -> size_t { // smallest t >= lo with B/q[t] <= t+16 (monotone)
    size_t a = lo, b = NQ; // invariant: answer in [a, b]
    while (a < b) { size_t mid = (a + b) / 2; if (B / (ld)q[mid] > (ld)(mid + 16)) a = mid + 1; else b
  = mid; }
    if (a == NQ) { fprintf(stderr, "PMAX too small\n"); exit(1); }
    return a;
};
// enumerate 47-smooth s with tau(s) | s, J(s) = v2(s/tau(s)) >= 1, and A/s > 15
const int P[15] = {2,3,5,7,11,13,17,19,23,29,31,37,41,43,47};
ld SMAXL = A / 15; ull SMAX = (ull)SMAXL;
vector<pair<ull,int>> good;
function<void(int, ull, ull)> dfs = [&](int i, ull s, ull tau) {
    if (i == 15) { if (s % tau == 0) { ull r = s / tau; int J = 0; while (r % 2 == 0) { r /= 2; J++; }
  if (J >= 1) good.push_back({s, J}); } return; }
    ull v = s; int e = 0;
    while (true) { dfs(i + 1, v, tau * (e + 1)); if (v > SMAX / P[i]) break; v *= P[i]; e++; }
};
dfs(0, 1, 1);
sort(good.begin(), good.end());
ld W[8] = {0}; ull E[8] = {0}; ull cnt[8] = {0};
for (auto &g : good) {
    ull s = g.first; int J = g.second;
    ld Bs = A / (ld)s;
    if (Bs > 15) { W[1] += 1.0L / (ld)s; E[1] += 15; cnt[1]++; }
    if (J >= 2 && JMAX >= 2) {
        size_t t2 = first_fail(Bs, 0);
        W[2] += inv[t2] / (ld)s; E[2] += pis[t2]; cnt[2] += t2;
    }
    if (J >= 3 && JMAX >= 3) {
        for (size_t t1 = 0; t1 < NQ; t1++) {
            ld B1 = Bs / (ld)q[t1];
            size_t c = first_fail(B1, t1 + 1);
            if (c == t1 + 1) break;
            W[3] += (inv[c] - inv[t1 + 1]) / ((ld)s * (ld)q[t1]); E[3] += pis[c] - pis[t1 + 1]; cnt[3]
  += c - t1 - 1;
        }
    }
    if (J >= 4 && JMAX >= 4) {
        for (size_t t1 = 0; t1 < NQ; t1++) {
            ld B1 = Bs / (ld)q[t1];
            bool any1 = false;
            for (size_t t2 = t1 + 1; t2 < NQ; t2++) {
                ld B2 = B1 / (ld)q[t2];
                size_t c = first_fail(B2, t2 + 1);
                if (c == t2 + 1) break;
                any1 = true;
                W[4] += (inv[c] - inv[t2 + 1]) / ((ld)s * (ld)q[t1] * (ld)q[t2]); E[4] += pis[c] - pis
 [t2 + 1]; cnt[4] += c - t2 - 1;
            }
            if (!any1) break;
        }
    }
    if (J >= 5 && JMAX >= 5) {
        for (size_t t1 = 0; t1 < NQ; t1++) {
            ld B1 = Bs / (ld)q[t1]; bool any1 = false;
            for (size_t t2 = t1 + 1; t2 < NQ; t2++) {
                ld B2 = B1 / (ld)q[t2]; bool any2 = false;
                for (size_t t3 = t2 + 1; t3 < NQ; t3++) {
                    ld B3 = B2 / (ld)q[t3];
                    size_t c = first_fail(B3, t3 + 1);
                    if (c == t3 + 1) break;
                    any2 = true;
                    W[5] += (inv[c] - inv[t3 + 1]) / ((ld)s * (ld)q[t1] * (ld)q[t2] * (ld)q[t3]); E[5]
  += pis[c] - pis[t3 + 1]; cnt[5] += c - t3 - 1;
```

```
                    }
                    if (!any2) break; any1 = true;
                }
                if (!any1) break;
            }
        }
    }
    ld Wt = 0; ull Et = 0;
    for (int j = 1; j <= 5; j++) { Wt += W[j]; Et += E[j]; printf("j=%d: terms=%llu  W_j=%.12Lf  E_j=%llu\
     n", j, cnt[j], W[j], E[j]); }
    ld D = 2 * Wt - 1 - 1.5L / lX;
    ld margin = A * D - 2.0L * (ld)Et;
    printf("X=%.6Le  log X=%.6Lf  A=X/logX=%.6Le  #good s=%zu\n", X, lX, A, good.size());
    printf("W=%.12Lf  2W=%.12Lf  needed 1+3/(2logX)=%.12Lf  D=%.12Lf\n", Wt, 2 * Wt, 1 + 1.5L / lX, D);
    printf("E=%llu  2E/A=%.12Lf\n", Et, 2.0L * (ld)Et / A);
    printf("margin A*D-2E = %.6Le  => %s\n", margin, (D > 0 && margin > 0) ? "TAIL CERTIFIED: pi(x) < 2T(x
     ) for all x >= X" : "NOT CERTIFIED");
    return 0;
}
```

B.8 Independent tail certificate (`tail_check.py`)

```
# Independent re-implementation of the tail certificate in Python.
# Template m = s * p_1 ... p_{j-1} (s 47-smooth, 2^j tau(s) | s, 53 <= p_1 < ... < p_{j-1} primes);
# included iff X > pi(p(m)) * m * log X, where p(m) = p_{j-1} (47 if j = 1).
import math, sys
import numpy as np
X = int(float(sys.argv[1])); PMAX = int(sys.argv[2])
logX = math.log(X); A = X / logX
sv = np.ones(PMAX + 1, dtype=bool); sv[:2] = False
for i in range(2, int(PMAX ** 0.5) + 1):
    if sv[i]: sv[i * i::i] = False
pr = np.nonzero(sv)[0]
q = [int(v) for v in pr[pr >= 53]]
NQ = len(q)
pis = [t + 16 for t in range(NQ)]                       # pi(q[t])
invpre = np.concatenate(([0.0], np.cumsum(1.0 / np.array(q, dtype=np.float64))))
pipre = [0]
for t in range(NQ): pipre.append(pipre[-1] + pis[t])
P = [2,3,5,7,11,13,17,19,23,29,31,37,41,43,47]
SMAX = int(A / 15) + 1
good = []
def dfs(i, s, tau):
    if i == 15:
        if s % tau == 0:
            r = s // tau; J = 0
            while r % 2 == 0: r //= 2; J += 1
            if J >= 1: good.append((s, J))
        return
    v, e = s, 0
    while True:
        dfs(i + 1, v, tau * (e + 1))
        if v * P[i] > SMAX: break
        v *= P[i]; e += 1
dfs(0, 1, 1)
def ok(m, pim): return X > pim * m * logX
def last_ok(base, lo):  # largest t >= lo with ok(base*q[t], pi(q[t])), or lo-1 if none (monotone in t)
    if not ok(base * q[lo], pis[lo]): return lo - 1
    a, b = lo, NQ - 1
    if ok(base * q[b], pis[b]): raise SystemExit("PMAX too small")
    while b - a > 1:
        mid = (a + b) // 2
        if ok(base * q[mid], pis[mid]): a = mid
        else: b = mid
    return a
W = [0.0] * 6; E = [0] * 6; nterm = [0] * 6
for (s, J) in good:
    if ok(s, 15): W[1] += 1.0 / s; E[1] += 15; nterm[1] += 1
    if J >= 2:
        c = last_ok(s, 0) + 1
        W[2] += invpre[c] / s; E[2] += pipre[c]; nterm[2] += c
```

```
    if J >= 3:
        for t1 in range(NQ - 1):
            c = last_ok(s * q[t1], t1 + 1) + 1
            if c == t1 + 1: break
            W[3] += (invpre[c] - invpre[t1 + 1]) / (s * q[t1]); E[3] += pipre[c] - pipre[t1 + 1]; nterm[3]
     += c - t1 - 1
    if J >= 4:
        for t1 in range(NQ - 2):
            any1 = False
            for t2 in range(t1 + 1, NQ - 1):
                c = last_ok(s * q[t1] * q[t2], t2 + 1) + 1
                if c == t2 + 1: break
                any1 = True
                W[4] += (invpre[c] - invpre[t2 + 1]) / (s * q[t1] * q[t2]); E[4] += pipre[c] - pipre[t2 +
    1]; nterm[4] += c - t2 - 1
            if not any1: break
Wt = sum(W); Et = sum(E); D = 2 * Wt - 1 - 1.5 / logX
for j in range(1, 5): print(f"j={j}: terms={nterm[j]} W_j={W[j]:.12f} E_j={E[j]}")
print(f"#good s={len(good)} W={Wt:.12f} E={Et} D={D:.12f} 2E/A={2*Et/A:.12f} margin={A*D-2*Et:.6e}",
      "CERTIFIED" if D > 0 and A * D - 2 * Et > 0 else "NOT CERTIFIED")
```

References

[1] S. Colton, *Refactorable numbers — a machine invention*, J. Integer Seq. **2** (1999), Article 99.1.2.

[2] M. Deléglise and J. Rivat, *Computing $\pi(x)$: the Meissel, Lehmer, Lagarias, Miller, Odlyzko method*, Math. Comp. **65** (1996), 235–245.

[3] Formal Conjectures (Google DeepMind and contributors), file `FormalConjectures/OEIS/111291.lean`, https://github.com/google-deepmind/formal-conjectures, commit `c094b97` (23 October 2026).

[4] X. Gourdon, *Computation of $\pi(x)$: improvements to the Meissel, Lehmer, Lagarias, Miller, Odlyzko, Deléglise and Rivat method*, preprint (2001).

[5] R. E. Kennedy and C. N. Cooper, *Tau numbers, natural density, and Hardy and Wright's Theorem 437*, Internat. J. Math. Math. Sci. **13** (1990), 383–386.

[6] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, sequences A033950 and A111291, https://oeis.org.

[7] J. B. Rosser and L. Schoenfeld, *Approximate formulas for some functions of prime numbers*, Illinois J. Math. **6** (1962), 64–94.

[8] C. Spiro, *How often is the number of divisors of n a divisor of n?*, J. Number Theory **21** (1985), 81–100.

[9] K. Walisch, *primecount* (version 7.10) and *primesieve* (version 12.0), https://github.com/kimwalisch/primecount.

[10] J. Zelinsky, *Tau numbers: a partial proof of a conjecture and other results*, J. Integer Seq. **5** (2002), Article 02.2.8.